

Tight Security of XMSS in the ROM (DRAFT 0.2.1)

Work in Progress

Abstract

We present a classical, multi-user, without corruption, proof of security of XMSS (eX-tended Merkle Signature Scheme) against *strong unforgeability under chosen message attack* (SUF-CMA), in the random oracle model.

Contents

1	Introduction	3
2	Definitions and Parameters	3
2.1	Basic Parameters	3
2.2	Tweakable Hash Function	3
3	Scheme Description	4
3.1	Message Encoding	4
3.2	WOTS Signature Scheme	4
3.2.1	Key Generation	5
3.2.2	Signature Generation	5
3.2.3	Signature Verification	5
3.3	Merkle Tree Construction	5
3.3.1	Leaf Computation	6
3.3.2	Internal Node Computation	6
3.3.3	Full Tree Construction	6
3.3.4	Authentication Path Computation	6
3.3.5	Root Computation from Authentication Path	7
3.4	XMSS Signature Scheme	7
3.4.1	XMSS Key Generation	7
3.4.2	XMSS Signature Structure	7
3.4.3	XMSS Signing	8
3.4.4	XMSS Verification	8
4	Security Model	8
5	Security Analysis	9
5.1	Adversary Model	9
5.2	No Public Parameter collide	10
5.3	All signature requests are made	10
5.4	All signature requests are valid	10
5.5	All random oracle queries are distinct	10
5.6	Classification of Forgery Events	11

6	Event $\mathcal{E}_{\text{enc}}$: Encoding Collision	12
6.1	Guessing Probability	12
6.2	Transition Probability	12
6.3	Final Bound	14
7	Event $\mathcal{E}_{\text{wots}}$: WOTS Forgery	14
7.1	Uniformity of Hidden Values	15
7.2	Guessing Probability	17
7.3	Transition Probability	17
7.4	Final Bound	18
8	Event $\mathcal{E}_{\text{leaf}}$: Merkle Leaf Collision	19
8.1	Guessing Probability	19
8.2	Transition Probability	19
8.3	Final Bound	20
9	Event $\mathcal{E}_{\text{merkle}}$: Merkle Internal Node Collision	20
9.1	Guessing Probability	20
9.2	Transition Probability	20
9.3	Final Bound	21
10	Main Result	21
11	Concrete Parameter Selection	22
11.1	150-bit security	22
11.2	128-bit security	23
12	TODO	23
A	Annex: Technical Results	23
A.1	Birthday Bound	23
A.2	Binomial Concentration	24

1 Introduction

XMSS is a hash-based, stateful, signature scheme that achieves security based solely on the properties of the underlying hash function. In this work, we analyze its concrete security in the random oracle model, proving strong unforgeability under chosen message attacks (SUF-CMA) in a multi-user setting.

Our analysis proceeds by classifying all possible forgery attacks into four (exhaustive) categories ($\mathcal{E}_{\text{enc}}$, $\mathcal{E}_{\text{wots}}$, $\mathcal{E}_{\text{leaf}}$, $\mathcal{E}_{\text{merkle}}$), then bounding the probability of each.

At the cost of relying on the Random Oracle Model, our proof brings practical improvements upon [1], [2], [3] (that rely purely on standard model assumptions).

2 Definitions and Parameters

We begin by establishing the notation and parameters used throughout this analysis.

2.1 Basic Parameters

- n : Number of bits in a hash digest.
- $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$: A hash function modeled as a (lazily sampled) random oracle.
- w : Hash chain length in WOTS (Winternitz One-Time Signature), assumed to be a power of 2.
- v : Number of hash chains in WOTS.
- pp : Number of bits for the public parameter—a random salt included in the XMSS public key and used in all hash function calls as protection against multi-user attacks.
- L : Lifetime, defined as the total number of signatures that can be generated with each XMSS key pair. The Merkle tree has height $h = \log_2(L)$, with L leaves indexed by an "epoch" ranging from 0 to $L - 1$.
- t : Target sum for the encoding scheme.
- r : Number of bits of randomness the signer generates before encoding.
- N : Number of signers in the multi-user setting.
- q : Total number of random oracle (H) queries made by the adversary.
- $\binom{[n]}{k}$: The set of all subsets of size k from the set $[1, n]$.

2.2 Tweakable Hash Function

Each invocation of the hash function H incorporates the public parameter PP alongside a *tweak*—which includes the domain separator and additional indices that uniquely identify the position of the hash computation within the scheme.

Definition 2.1 (Domain Separator Constants). The scheme uses four domain separator values to distinguish hash contexts:

- $\text{D}_{\text{ENC}} = 0$: Hash computations for message encoding.
- $\text{D}_{\text{CHAIN}} = 1$: Hash computations within WOTS hash chains.
- $\text{D}_{\text{LEAF}} = 2$: Hash computations for WOTS public key compression (Merkle tree leaves).
- $\text{D}_{\text{TREE}} = 3$: Hash computations for internal Merkle tree nodes.

3 Scheme Description

3.1 Message Encoding

To sign a message $m \in \{0, 1\}^{256}$ at epoch $e < L$, the signer computes an encoding $(x_0, \dots, x_{v-1}) \in [0, w-1]^v$ via a randomized procedure that ensures a fixed target sum. We first define the base encoding function, then the full encoding procedure.

Definition 3.1 (Base Encoding Function). The function $\text{enc} : \{0, 1\}^n \rightarrow [0, w-1]^v$ interprets the first $v \cdot \log_2 w$ bits of its input as a sequence of v integers in $[0, w-1]$. Specifically, for $y \in \{0, 1\}^n$:

$$\text{enc}(y) = (x_0, x_1, \dots, x_{v-1}) \quad \text{where } x_i = \sum_{j=0}^{\log_2 w - 1} y[i \cdot \log_2 w + j] \cdot 2^j$$

Algorithm 1: Encode(PP, e, m): Message Encoding

Input: Public parameter $\text{PP} \in \{0, 1\}^{\text{PP}}$, epoch $e \in [0, L)$, message $m \in \{0, 1\}^{256}$

Output: Randomness $R \in \{0, 1\}^r$ and encoding $(x_0, \dots, x_{v-1}) \in [0, w-1]^v$

Parameters: Target sum t , randomness size r

```

1  $\mathcal{S} \leftarrow \{0, 1\}^r$  ; // Set of available randomness values
2 while  $\mathcal{S} \neq \emptyset$  do
3   Sample  $R \xleftarrow{\$} \mathcal{S}$  uniformly at random;
4    $\mathcal{S} \leftarrow \mathcal{S} \setminus \{R\}$ ;
5    $y \leftarrow H(\text{PP} \parallel \text{D}_{\text{ENC}} \parallel e \parallel R \parallel m)$ ;
6    $(x_0, x_1, \dots, x_{v-1}) \leftarrow \text{enc}(y)$ ;
7   if  $\sum_{i=0}^{v-1} x_i = t$  then
8     return  $(R, (x_0, \dots, x_{v-1}))$ ;
9 return  $\perp$  ; // Abort: no valid encoding found
```

Definition 3.2 (Encoding Probability). Let P_{enc} denote the probability that a uniformly random sequence of v values from $\{0, 1, \dots, w-1\}$ satisfies the target sum constraint $\sum_{i=0}^{v-1} x_i = t$. On average, generating valid randomness requires $1/P_{\text{enc}}$ attempts.

Remark 3.3 (Incomparability of Encodings). The target sum constraint ensures that valid encodings are *incomparable*: given two *different* valid encodings $(x_0, \dots, x_{v-1})$ and $(x'_0, \dots, x'_{v-1})$, there must exist indices j and k such that $x_j < x'_j$ and $x_k > x'_k$.

3.2 WOTS Signature Scheme

WOTS (Winternitz One-Time Signature) is a one time signature (OTS) scheme.

Definition 3.4 (Chain Function). For public parameter PP , epoch e , chain index $j \in [0, v)$, start position a , end position b (with $0 \leq a \leq b \leq w-1$), and input $x \in \{0, 1\}^n$, define:

$$\text{chain}(\text{PP}, e, j, a, b, x) = \begin{cases} x & \text{if } a = b \\ H(\text{PP} \parallel \text{D}_{\text{CHAIN}} \parallel e \parallel j \parallel b - 1 \parallel \text{chain}(\text{PP}, e, j, a, b - 1, x)) & \text{if } a < b \end{cases}$$

Equivalently, $\text{chain}(\text{PP}, e, j, a, b, x)$ applies $b - a$ hash iterations to x , with tweaks incorporating the position index.

3.2.1 Key Generation

Algorithm 2: WOTS.KeyGen(PP, e): WOTS Key Generation

Input: Public parameter PP $\in \{0, 1\}^{\text{PP}}$, epoch $e \in [0, L)$

Output: Secret key $\text{sk} \in (\{0, 1\}^n)^v$, public key $\text{pk} \in (\{0, 1\}^n)^v$

```

1 for  $j \leftarrow 0$  to  $v - 1$  do
2    $\text{sk}_j \xleftarrow{\$} \{0, 1\}^n$ ; // Sample secret key uniformly at random
3    $c_{j,0} \leftarrow \text{sk}_j$ ;
4   for  $k \leftarrow 0$  to  $w - 2$  do
5      $c_{j,k+1} \leftarrow H(\text{PP} \parallel \text{D}_{\text{CHAIN}} \parallel e \parallel j \parallel k \parallel c_{j,k})$ ;
6    $\text{pk}_j \leftarrow c_{j,w-1}$ ;
7 return  $(\text{sk} = (\text{sk}_0, \dots, \text{sk}_{v-1}), \text{pk} = (\text{pk}_0, \dots, \text{pk}_{v-1}))$ 

```

Remark 3.5 (Chain Values). We denote by $c_{j,k}$ the *chain value* at position k in chain j . Note that $c_{j,0} = \text{sk}_j$ (the secret key element) and $c_{j,w-1} = \text{pk}_j$ (the public key element). Using 3.4: $c_{j,k} = \text{chain}(\text{PP}, e, j, 0, k, \text{sk}_j)$.

3.2.2 Signature Generation

Algorithm 3: WOTS.Sign(PP, e , sk , $(x_0, \dots, x_{v-1})$): WOTS Signing

Input: Public parameter PP, epoch e , secret key $\text{sk} = (\text{sk}_0, \dots, \text{sk}_{v-1})$, encoding $(x_0, \dots, x_{v-1}) \in [0, w - 1]^v$

Output: Signature $\text{sig} = (\text{sig}_0, \dots, \text{sig}_{v-1}) \in (\{0, 1\}^n)^v$

```

1 for  $j \leftarrow 0$  to  $v - 1$  do
2    $\text{sig}_j \leftarrow \text{chain}(\text{PP}, e, j, 0, x_j, \text{sk}_j)$ ; //  $\text{sig}_j = c_{j,x_j}$ 
3 return  $\text{sig} = (\text{sig}_0, \dots, \text{sig}_{v-1})$ 

```

3.2.3 Signature Verification

Algorithm 4: WOTS.RecoverPK(PP, e , $(x_0, \dots, x_{v-1})$, sig): Recover WOTS Public Key

Input: Public parameter PP, epoch e , encoding $(x_0, \dots, x_{v-1})$, signature $\text{sig} = (\text{sig}_0, \dots, \text{sig}_{v-1})$

Output: Reconstructed public key $\text{pk}' = (\text{pk}'_0, \dots, \text{pk}'_{v-1})$

```

1 for  $j \leftarrow 0$  to  $v - 1$  do
2    $\text{pk}'_j \leftarrow \text{chain}(\text{PP}, e, j, x_j, w - 1, \text{sig}_j)$ ; // Complete the chain from  $\text{sig}_j$ 
3 return  $\text{pk}' = (\text{pk}'_0, \dots, \text{pk}'_{v-1})$ 

```

Algorithm 5: WOTS.Verify(PP, e , pk , $(x_0, \dots, x_{v-1})$, sig): WOTS Verification

Input: Public parameter PP, epoch e , public key pk , encoding $(x_0, \dots, x_{v-1})$, signature sig

Output: 1 (accept) or 0 (reject)

```

1  $\text{pk}' \leftarrow \text{WOTS.RecoverPK}(\text{PP}, e, (x_0, \dots, x_{v-1}), \text{sig})$ ;
2 if  $\text{pk}' = \text{pk}$  then return 1;
3 else return 0;

```

3.3 Merkle Tree Construction

The Merkle tree has $L = 2^h$ leaves (one per epoch) and height h . We use the convention that depth 0 is the leaf level and depth h is the root.

3.3.1 Leaf Computation

Each leaf is the hash of a WOTS public key, binding the public key to its epoch index.

Algorithm 6: ComputeLeaf(PP, e , $\mathbf{pk}$): Merkle Leaf Computation

Input: Public parameter PP, epoch $e \in [0, L)$, WOTS public key $\mathbf{pk} = (\mathbf{pk}_0, \dots, \mathbf{pk}_{v-1})$

Output: Leaf hash $\text{leaf} \in \{0, 1\}^n$

1 return $H(\text{PP} \parallel \text{D}_{\text{LEAF}} \parallel e \parallel \mathbf{pk}_0 \parallel \mathbf{pk}_1 \parallel \dots \parallel \mathbf{pk}_{v-1})$

3.3.2 Internal Node Computation

Internal nodes are computed by hashing their two children together with the depth and position index.

Algorithm 7: ComputeNode(PP, d, j, A, B): Merkle Internal Node

Input: Public parameter PP, depth $d \in [1, h]$, index $j \in [0, 2^{h-d})$, left child $A \in \{0, 1\}^n$, right child $B \in \{0, 1\}^n$

Output: Internal node hash $\in \{0, 1\}^n$

1 return $H(\text{PP} \parallel \text{D}_{\text{TREE}} \parallel d \parallel j \parallel A \parallel B)$

3.3.3 Full Tree Construction

Algorithm 8: BuildTree(PP, $(\mathbf{pk}^{(0)}, \dots, \mathbf{pk}^{(L-1)})$): Merkle Tree Construction

Input: Public parameter PP, WOTS public keys $\mathbf{pk}^{(e)}$ for all epochs $e \in [0, L)$

Output: Root $\text{root} \in \{0, 1\}^n$ and all internal nodes $\text{node}[d][j]$

// Compute leaves (depth 0)

1 for $e \leftarrow 0$ to $L - 1$ do

2 | $\text{node}[0][e] \leftarrow \text{ComputeLeaf}(\text{PP}, e, \mathbf{pk}^{(e)});$

// Compute internal nodes bottom-up

3 for $d \leftarrow 1$ to h do

4 | for $j \leftarrow 0$ to $2^{h-d} - 1$ do

5 | | $A \leftarrow \text{node}[d-1][2j];$ // Left child

6 | | $B \leftarrow \text{node}[d-1][2j+1];$ // Right child

7 | | $\text{node}[d][j] \leftarrow \text{ComputeNode}(\text{PP}, d, j, A, B);$

8 $\text{root} \leftarrow \text{node}[h][0];$

9 return (root, node)

3.3.4 Authentication Path Computation

The authentication path for epoch e consists of the sibling nodes along the path from leaf e to the root.

Algorithm 9: AuthPath(e, node): Compute Authentication Path

Input: Epoch $e \in [0, L)$, precomputed tree nodes node

Output: Authentication path $(\text{auth}_0, \dots, \text{auth}_{h-1}) \in (\{0, 1\}^n)^h$

```
1 idx  $\leftarrow e$  ; // Current index at depth 0
2 for  $d \leftarrow 0$  to  $h - 1$  do
    // Sibling is at index  $\text{idx} \oplus 1$  (flip last bit)
3     sibling_idx  $\leftarrow \text{idx} \oplus 1$ ;
4      $\text{auth}_d \leftarrow \text{node}[d][\text{sibling\_idx}]$ ;
5      $\text{idx} \leftarrow \lfloor \text{idx}/2 \rfloor$  ; // Move to parent index
6 return  $(\text{auth}_0, \dots, \text{auth}_{h-1})$ 
```

3.3.5 Root Computation from Authentication Path

Given a leaf and its authentication path, compute the Merkle root.

Algorithm 10: ComputeRoot($\text{PP}, e, \text{leaf}, (\text{auth}_0, \dots, \text{auth}_{h-1})$): Compute Root from Path

Input: Public parameter PP , epoch e , leaf value $\text{leaf} \in \{0, 1\}^n$, authentication path $(\text{auth}_0, \dots, \text{auth}_{h-1})$

Output: Computed root $\text{root}' \in \{0, 1\}^n$

```
1 current  $\leftarrow \text{leaf}$ ;
2 idx  $\leftarrow e$ ;
3 for  $d \leftarrow 0$  to  $h - 1$  do
4      $j \leftarrow \lfloor \text{idx}/2 \rfloor$ ;
5     if  $\text{idx} \bmod 2 = 0$  then current  $\leftarrow \text{ComputeNode}(\text{PP}, d + 1, j, \text{current}, \text{auth}_d)$ ;
6     else current  $\leftarrow \text{ComputeNode}(\text{PP}, d + 1, j, \text{auth}_d, \text{current})$ ;
7      $\text{idx} \leftarrow j$ ;
8 return current
```

3.4 XMSS Signature Scheme

XMSS combines WOTS with a Merkle tree to enable L signatures per key pair.

3.4.1 XMSS Key Generation

Algorithm 11: XMSS.KeyGen: XMSS Key Generation

Output: Public key PK , secret key SK

```
1  $\text{PP} \xleftarrow{\$} \{0, 1\}^{\text{PP}}$  ; // Sample public parameter
    // Generate WOTS key pairs for all epochs
2 for  $e \leftarrow 0$  to  $L - 1$  do
3      $(\text{sk}^{(e)}, \text{pk}^{(e)}) \leftarrow \text{WOTS.KeyGen}(\text{PP}, e)$ ;
    // Build Merkle tree
4  $(\text{root}, \text{node}) \leftarrow \text{BuildTree}(\text{PP}, (\text{pk}^{(0)}, \dots, \text{pk}^{(L-1)}))$ ;
5  $\text{PK} \leftarrow (\text{PP}, \text{root})$ ;
6  $\text{SK} \leftarrow (\text{PP}, (\text{sk}^{(0)}, \dots, \text{sk}^{(L-1)}), \text{node})$  ; // Store tree for auth paths
7 return  $(\text{PK}, \text{SK})$ 
```

3.4.2 XMSS Signature Structure

An XMSS signature for message $m \in \{0, 1\}^{256}$ at epoch e consists of:

1. Randomness $R \in \{0, 1\}^r$ used for encoding.
2. WOTS signature $\text{sig} = (\text{sig}_0, \text{sig}_1, \dots, \text{sig}_{v-1}) \in (\{0, 1\}^n)^v$.
3. Authentication path $(\text{auth}_0, \text{auth}_1, \dots, \text{auth}_{h-1}) \in (\{0, 1\}^n)^h$.

3.4.3 XMSS Signing

Algorithm 12: XMSS.Sign(SK, e, m): XMSS Signing

Input: Secret key $\text{SK} = (\text{PP}, (\text{sk}^{(0)}, \dots, \text{sk}^{(L-1)}), \text{node})$, epoch $e \in [0, L)$, message $m \in \{0, 1\}^{256}$

Output: Signature $\sigma = (R, \text{sig}, (\text{auth}_0, \dots, \text{auth}_{h-1}))$ or $\perp$

// Step 1: Compute encoding

- 1 $(R, (x_0, \dots, x_{v-1})) \leftarrow \text{Encode}(\text{PP}, e, m)$;
- 2 **if** $R = \perp$ **then return** $\perp$;

// Step 2: Generate WOTS signature

- 3 $\text{sig} \leftarrow \text{WOTS.Sign}(\text{PP}, e, \text{sk}^{(e)}, (x_0, \dots, x_{v-1}))$;

// Step 3: Compute authentication path

- 4 $(\text{auth}_0, \dots, \text{auth}_{h-1}) \leftarrow \text{AuthPath}(e, \text{node})$;
- 5 **return** $\sigma = (R, \text{sig}, (\text{auth}_0, \dots, \text{auth}_{h-1}))$

3.4.4 XMSS Verification

Algorithm 13: XMSS.Verify(PK, e, m, σ): XMSS Verification

Input: Public key $\text{PK} = (\text{PP}, \text{root})$, epoch e , message m , signature $\sigma = (R, \text{sig}, \text{auth})$

Output: 1 (accept) or 0 (reject)

// Step 1: Recompute encoding

- 1 $(x_0, \dots, x_{v-1}) \leftarrow \text{enc}(H(\text{PP} \parallel \text{D}_{\text{ENC}} \parallel e \parallel R \parallel m))$;
- 2 **if** $\sum_{i=0}^{v-1} x_i \neq t$ **then return** 0;

// Step 2: Reconstruct WOTS public key

- 3 $\text{pk}' \leftarrow \text{WOTS.RecoverPK}(\text{PP}, e, (x_0, \dots, x_{v-1}), \text{sig})$;

// Step 3: Compute leaf and root

- 4 $\text{leaf}' \leftarrow \text{ComputeLeaf}(\text{PP}, e, \text{pk}')$;
- 5 $\text{root}' \leftarrow \text{ComputeRoot}(\text{PP}, e, \text{leaf}', \text{auth})$;

// Step 4: Compare roots

- 6 **if** $\text{root}' = \text{root}$ **then return** 1;
- 7 **else return** 0;

4 Security Model

We prove security in the *strong unforgeability under chosen message attack* (SUF-CMA) model in a multi-user setting.

Definition 4.1 (Multi-User SUF-CMA Security Game). Let $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify})$ be a stateful signature scheme and let $N \in \mathbb{N}$ be the number of signers. The multi-user SUF-CMA game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ proceeds as follows:

1. **Setup.** The challenger generates N independent key pairs:

$$(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(1^\lambda) \quad \text{for } i \in \{0, \dots, N-1\}$$

and gives $(\mathbf{pk}_0, \dots, \mathbf{pk}_{N-1})$ to the adversary. The challenger initializes $\mathcal{Q}_{i,e} \leftarrow \emptyset$ for all $i \in \{0, \dots, N-1\}$ and $e \in \{0, \dots, L-1\}$.

2. **Query phase.** The adversary adaptively interleaves the following queries:

- *Random oracle query:* $\mathcal{A}$ queries $H(x)$ for any input $x \in \{0, 1\}^*$ and receives the output.
- *Signing query:* $\mathcal{A}$ queries (i, e, m) where $i \in \{0, \dots, N-1\}$, $e \in \{0, \dots, L-1\}$, and $m \in \{0, 1\}^{256}$. If $\mathcal{Q}_{i,e} \neq \emptyset$ (epoch already used for signer i), the query is rejected (and the game aborts). Otherwise, the challenger computes $\sigma \leftarrow \text{Sign}(\mathbf{sk}_i, e, m)$, sets $\mathcal{Q}_{i,e} \leftarrow \{(m, \sigma)\}$, and returns σ to $\mathcal{A}$.

Remark 4.2. When computing the signature, the challenger iteratively picks at random, uniformly and without repetitions, $R \in \{0, 1\}^r$ until a valid encoding is found. If after all the 2^r choices of R no valid encoding is found, the game aborts.

3. **Forgery.** The adversary outputs (i, e, m^*, σ^*) .

4. **Winning condition.** The adversary *wins* if:

- (a) $\text{Verify}(\mathbf{pk}_i, e, m^*, \sigma^*) = 1$, and
- (b) $(m^*, \sigma^*) \notin \mathcal{Q}_{i,e}$.

Remark 4.3 (Strong Unforgeability). Condition (b) captures *strong* unforgeability: the adversary wins even if m^* was previously signed at epoch e by signer i , provided σ^* differs from the signature previously returned. This is stronger than standard unforgeability, which only requires the message to be new.

Remark 4.4 (Maximum Signatures). The total number of signatures across all signers is at most $L \cdot N$.

Definition 4.5 (Signer random variables). For $i \in \{0, \dots, N-1\}$, we define:

- PP_i : the public parameter of signer i .
- For $e \in [0, L), j \in [0, v), k \in [0, w)$: $c_{i,e,j,k} = \text{chain}(\text{PP}_i, e, j, 0, k, \mathbf{sk}_j^{(e)})$ is the chain value at position k in chain j for signer i at epoch e (as computed in 2).

Definition 4.6. At any point in the game, we denote by *fresh* a random oracle query if it has not been made before (either by the adversary or the challenger).

5 Security Analysis

5.1 Adversary Model

We consider a classical (non-quantum) adversary $\mathcal{A}$. We model the adversary as a deterministic algorithm; this assumption is without loss of generality in the random oracle model, as any randomized adversary can be derandomized by fixing its random tape to its most successful value. The probability that $\mathcal{A}$ wins the SUF-CMA game is denoted $\Pr[\mathcal{A} \text{ wins}]$ (where we omit "SUF-CMA" for brevity).

We denote by q the total number of (classical) random oracle queries made by the adversary.

For $s \in [0, q + LN]$, we define the random variable $\mathcal{Q}(s)$ as the set of the first s queries (counting together random oracle queries and signature requests) made by the adversary.

Each query in $\mathcal{Q}(s)$ is either:

- a random oracle query of the form $\text{ROquery}(x)$ for some data $x \in \{0, 1\}^*$

- a signature request of the form $\text{SigRequest}(i, e, m)$ for some signer $i \in [0, N)$, epoch $e \in [0, L)$, and message $m \in \{0, 1\}^{256}$.

For simplicity, we denote by $\mathcal{Q} = \mathcal{Q}(q + LN)$ the set of all queries made by the adversary during the game.

5.2 No Public Parameter collide

Let us denote by $\mathcal{E}_{\text{distinct PP}}$ the event: "all public parameters PP across the N signers are distinct".

Proposition 5.1. $\Pr[\neg \mathcal{E}_{\text{distinct PP}}] \leq \frac{N^2}{2^{\text{pp}} + 1}$

Proof. Each signer samples $PP \in \{0, 1\}^{\text{pp}}$ independently and uniformly at random. By Theorem A.1 with $k = N$ values sampled from a set of size 2^{pp} , the probability of any collision is at most $N^2 / (2 \cdot 2^{\text{pp}}) = N^2 / 2^{\text{pp}+1}$. $\square$

Assumption: We condition the rest of our analysis on $\mathcal{E}_{\text{distinct PP}}$, i.e. we will focus on bounding $\max_{\mathcal{A}} \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_{\text{distinct PP}}]$. Note that $\mathcal{E}_{\text{distinct PP}}$ is independent from the adversary.

5.3 All signature requests are made

Let $\mathcal{E}_0$ denotes the event: $\mathcal{E}_{\text{distinct PP}} \cap$ "all the LN signature requests are made by the adversary". Given any adversary $\mathcal{A}$, it's possible to construct another adversary $\mathcal{A}'$ that always respects $\mathcal{E}_0$, by querying the m missing (signer, epoch) with the zero message, just before $\mathcal{A}$ outputs its forgery. Except with probability $\Pr[\text{R not found}] \leq m(1 - P_{\text{enc}})^{2^r} \leq LN(1 - P_{\text{enc}})^{2^r}$, these m additional signing requests do not lead to "abort" (c.f. 4.2). As a result:

$$\begin{aligned} \max_{\mathcal{A}} \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_{\text{distinct PP}}] &\leq \max_{\mathcal{A}} \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_0] + \Pr[\text{R not found}] \\ &\leq \max_{\mathcal{A}} \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_0] + LN(1 - P_{\text{enc}})^{2^r} \end{aligned}$$

5.4 All signature requests are valid

Let $\mathcal{E}_1$ denotes the event: $\mathcal{E}_0 \cap$ "all signature requests concern distinct (signer, epoch) pairs". Given any adversary $\mathcal{A}$, it's possible to construct another adversary $\mathcal{A}'$ that always respects $\mathcal{E}_1$, by skipping every duplicate request (that would otherwise lead to "abort"), which is always a better strategy. As a result:

$$\max_{\mathcal{A}} \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_0] \leq \max_{\mathcal{A}} \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_1]$$

5.5 All random oracle queries are distinct

Let $\mathcal{E}_2$ denotes the event: $\mathcal{E}_1 \cap$ "all random oracle queries are distinct". Given any adversary $\mathcal{A}$, it's possible to construct another adversary $\mathcal{A}'$ that always respects $\mathcal{E}_2$, because H input space $= \{0, 1\}^*$ is infinite, without influencing the probability of winning. As a result:

$$\max_{\mathcal{A}} \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_1] = \max_{\mathcal{A}} \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_2]$$

5.6 Classification of Forgery Events

Assume $\mathcal{E}_2$. Suppose the adversary produces a valid forgery $\sigma^* = (R^*, \text{sig}^*, \text{auth}^*)$ for signer i , at epoch e , on message m^* .

Let m be the message requested at epoch e to signer i (recall $\mathcal{E}_2 \subset \mathcal{E}_0$), and let $\sigma = (R, \text{sig}, \text{auth})$ denote the corresponding signature. We define:

- Encodings (via Algorithm 1): $(x_0, \dots, x_{v-1}) = \text{enc}(H(\text{PP} \parallel \text{D}_{\text{ENC}} \parallel e \parallel R \parallel m))$ and $(x_0^*, \dots, x_{v-1}^*)$ similarly for the forgery.
- WOTS public keys (via Algorithm 4): $\text{pk} = \text{WOTS.RecoverPK}(\text{PP}, e, (x_0, \dots, x_{v-1}), \text{sig})$ and pk^* similarly.
- Merkle leaves (via Algorithm 6): $\text{leaf} = \text{ComputeLeaf}(\text{PP}, e, \text{pk})$ and leaf^* similarly.
- Sibling pairs: $((\text{left}_d, \text{right}_d))_{d=0}^{h-1}$ and $((\text{left}_d^*, \text{right}_d^*))_{d=0}^{h-1}$ from the authentication paths.
- Chain values (via Algorithm 2): $(c_{i,e,j,k})_{j \in [0,v], k \in [0,w]}$ for signer i at epoch e .

Given $(m^*, \sigma^*) \neq (m, \sigma)$, at least one of the following events must occur:

$\mathcal{E}_{\text{enc}}$ (**Encoding Collision**): $\mathcal{E}_2$ holds and $(R^*, m^*) \neq (R, m)$ but the encodings are identical:

$$(x_0^*, \dots, x_{v-1}^*) = (x_0, \dots, x_{v-1})$$

$\mathcal{E}_{\text{wots}}$ (**WOTS Forgery**): $\mathcal{E}_2$ holds and $\text{pk}^* = \text{pk}$ but $(x^*, \text{sig}^*) \neq (x, \text{sig})$, i.e., WOTS.RecoverPK returns the same public key from different (encoding, signature) pairs.

$\mathcal{E}_{\text{leaf}}$ (**Merkle Leaf Collision**): $\mathcal{E}_2$ holds and $\text{pk}^* \neq \text{pk}$ but $\text{ComputeLeaf}(\text{PP}, e, \text{pk}^*) = \text{ComputeLeaf}(\text{PP}, e, \text{pk})$.

$\mathcal{E}_{\text{merkle}}$ (**Merkle Internal Node Collision**): $\mathcal{E}_2$ holds and $\exists d \in [0, h-1]$ such that $(\text{left}_d^*, \text{right}_d^*) \neq (\text{left}_d, \text{right}_d)$ but $\text{ComputeNode}(\text{PP}, d+1, j, \text{left}_d^*, \text{right}_d^*) = \text{ComputeNode}(\text{PP}, d+1, j, \text{left}_d, \text{right}_d)$ where $j = \lfloor e/2^{d+1} \rfloor$.

Proposition 5.2 (Exhaustiveness). *For any adversary $\mathcal{A}$:*

$$[\mathcal{A} \text{ wins} \mid \mathcal{E}_2] = [\mathcal{A} \text{ wins} \mid \mathcal{E}_{\text{enc}}] \cup [\mathcal{A} \text{ wins} \mid \mathcal{E}_{\text{wots}}] \cup [\mathcal{A} \text{ wins} \mid \mathcal{E}_{\text{leaf}}] \cup [\mathcal{A} \text{ wins} \mid \mathcal{E}_{\text{merkle}}]$$

Proof. Assume by contradiction that none of $\mathcal{E}_{\text{enc}}$, $\mathcal{E}_{\text{wots}}$, $\mathcal{E}_{\text{leaf}}$, or $\mathcal{E}_{\text{merkle}}$ occurs.

Identical Merkle path:

$$\neg \mathcal{E}_{\text{merkle}} \implies (\text{auth}_0^*, \dots, \text{auth}_{h-1}^*) = (\text{auth}_0, \dots, \text{auth}_{h-1})$$

Identical WOTS Public Keys:

Suppose for contradiction that the WOTS public keys differ. Then $\neg \mathcal{E}_{\text{leaf}} \implies \text{leaf}^* \neq \text{leaf}$. Starting from these different leaves at depth h , both authentication paths lead to the same root. Let d' be the smallest depth at which the Merkle nodes first coincide. Then $\mathcal{E}_{\text{merkle}}$ holds for $d = d' - 1$, contradiction.

Identical WOTS signature:

Since the WOTS public keys are identical, and $\neg \mathcal{E}_{\text{wots}}$, we must have:

$$(x_0^*, \dots, x_{v-1}^*, \text{sig}_0^*, \dots, \text{sig}_{v-1}^*) = (x_0, \dots, x_{v-1}, \text{sig}_0, \dots, \text{sig}_{v-1})$$

Identical Randomness and Message:

Since the encodings are identical, and $\neg \mathcal{E}_{\text{enc}}$, we must have $(R^*, m^*) = (R, m)$.

Contradiction:

We have $(m, \sigma) = (m^*, \sigma^*)$, contradicting the assumption that the adversary wins the SUF-CMA game. □

Definition 5.3. We index the request of the adversary by $[1, q + LN]$. For $\text{ord} \in \binom{[q+LN]}{q}$, we define the event $\mathcal{E}_{\text{ord}}$: "The s -th query of the adversary is a random oracle query if and only if $s \in \text{ord}$ ".

Lemma 5.4. Given $\mathcal{E}_2$, any winning adversary makes exactly q random oracle queries and LN signature requests. Thus $(\mathcal{E}_{\text{ord}})_{\text{ord}}$ form a partition of $\mathcal{E}_2 \supset \mathcal{E}_{\text{enc}}^{\text{found}}$.

6 Event $\mathcal{E}_{\text{enc}}$: Encoding Collision

Consider the following events:

- For $s \in [0, q + LN]$, $\mathcal{E}_{\text{enc}}^{\text{found}}(s) = \mathcal{E}_2 \cap$ "There exists in $\mathcal{Q}(s)$ a random oracle query of the form $(\text{PP}_i \| \text{D}_{\text{ENC}} \| e \| R^* \| m^*)$ and a signature request (i, e, m) , with public parameter PP and randomness R , such that $(R^*, m^*) \neq (R, m)$ but the encodings are identical: $\text{enc}(H(\text{PP} \| \text{D}_{\text{ENC}} \| e \| R \| m)) = \text{enc}(H(\text{PP} \| \text{D}_{\text{ENC}} \| e \| R^* \| m^*))$ ".

For simplicity, we denote $\mathcal{E}_{\text{enc}}^{\text{found}} = \mathcal{E}_{\text{enc}}^{\text{found}}(q + LN)$.

- $\mathcal{E}_{\text{enc}}^{\text{guessed}} = \mathcal{E}_{\text{enc}} \setminus \mathcal{E}_{\text{enc}}^{\text{found}}$

6.1 Guessing Probability

Lemma 6.1. $\Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{enc}}^{\text{guessed}}] \leq 1/w^v$.

Proof. Assuming $\mathcal{E}_{\text{enc}}^{\text{guessed}}$, the encoding from the forged signature leads to a *fresh* random oracle query (4.6). As a result, $H(\text{PP} \| \text{D}_{\text{ENC}} \| e \| R^* \| m^*)$ is uniform in $\{0, 1\}^n$, and so $\text{enc}(H(\text{PP} \| \text{D}_{\text{ENC}} \| e \| R^* \| m^*))$ is uniform in $[0, w - 1]^v$. The probability that it matches the encoding from the signature request is therefore $1/w^v$. $\square$

6.2 Transition Probability

Definition 6.2. We denote: $\mathcal{E}_{\text{enc}}^{\text{transition}}(s) = \neg \mathcal{E}_{\text{enc}}^{\text{found}}(s) \cap \mathcal{E}_{\text{enc}}^{\text{found}}(s + 1)$ for $s \in [0, q + LN]$.

Lemma 6.3. $\mathcal{E}_{\text{enc}}^{\text{found}} = \bigcup_{s=0}^{q+LN-1} \mathcal{E}_{\text{enc}}^{\text{transition}}(s)$

Proof. $\mathcal{E}_{\text{enc}}^{\text{found}}(0) = \emptyset$ and $\forall s \in [0, q + LN - 1], \mathcal{E}_{\text{enc}}^{\text{found}}(s) \subset \mathcal{E}_{\text{enc}}^{\text{found}}(s + 1)$. $\square$

Lemma 6.4.

$$\Pr[\mathcal{E}_{\text{enc}}^{\text{found}}] \leq \sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\mathcal{E}_{\text{ord}}] \cdot \sum_{s=0}^{q+LN-1} \Pr[\mathcal{E}_{\text{enc}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}]$$

Proof. By 5.4, we have:

$$\begin{aligned}
\Pr[\mathcal{E}_{\text{enc}}^{\text{found}}] &= \sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\mathcal{E}_{\text{ord}} \cap \mathcal{E}_{\text{enc}}^{\text{found}}] \\
&= \sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\mathcal{E}_{\text{ord}} \cap \bigcup_{s=0}^{q+LN-1} \mathcal{E}_{\text{enc}}^{\text{transition}}(s)] \\
&= \sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\bigcup_{s=0}^{q+LN-1} [\mathcal{E}_{\text{ord}} \cap \mathcal{E}_{\text{enc}}^{\text{transition}}(s)]] \\
&\leq \sum_{\text{ord} \in \binom{[q+LN]}{q}} \sum_{s=0}^{q+LN-1} \Pr[\mathcal{E}_{\text{ord}} \cap \mathcal{E}_{\text{enc}}^{\text{transition}}(s)] \\
&= \sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\mathcal{E}_{\text{ord}}] \cdot \sum_{s=0}^{q+LN-1} \Pr[\mathcal{E}_{\text{enc}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}]
\end{aligned}$$

□

Fix $\text{ord} \in \binom{[q+LN]}{q}$, $s \in [0, q+LN-1]$ and assume $\mathcal{E}_2 \cap \neg \mathcal{E}_{\text{enc}}^{\text{found}}(s) \cap \mathcal{E}_{\text{ord}}$ holds.

If $s+1 \in \text{ord}$ (Random Oracle Query)

For $\mathcal{E}_{\text{enc}}^{\text{found}}(s+1)$ to be triggered, the $(s+1)^{\text{th}}$ query should have the form $\text{ROQuery}(\text{PP} \parallel \text{D}_{\text{ENC}} \parallel e \parallel R \parallel m)$, and it must be a *fresh* (4.6) query from the oracle's perspective. As a result, the oracle's answer is uniform in $\{0, 1\}^n$, and thus the corresponding encoding $\text{enc}(H(\text{PP} \parallel \text{D}_{\text{ENC}} \parallel e \parallel R \parallel m))$ is uniform in $[0, w-1]^v$. Since there is at most one signed encoding per (signer, epoch) pair, the probability of collision is at most $1/w^v$.

Conclusion: Assuming $s+1 \in \text{ord}$, $\Pr[\mathcal{E}_{\text{enc}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] \leq 1/w^v$.

Otherwise $s+1 \notin \text{ord}$ (Signature Request)

Let (i, PP, e, m) respectively denote the signer index, public parameter, epoch and message corresponding to the $(s+1)^{\text{th}}$ query (which is a signature request).

Let C_s denote the number of previous queries of the form $\text{ROQuery}(\text{PP} \parallel \text{D}_{\text{ENC}} \parallel e \parallel \cdot \parallel \cdot)$, and let C_s^+ denote the number among these that yield **valid encodings**.

The adversary has queried C_s values of R , leaving $2^r - C_s$ unqueried choices. Among the unqueried choices, the number yielding valid encodings is a random variable. Specifically, conditioned on the adversary's queries, the number of unqueried R values yielding valid encodings follows a binomial distribution, which we can bound using concentration inequalities.

By Theorem A.2 applied to the $2^r - C_s$ unqueried values (each yielding a valid encoding with probability P_{enc}), except with probability at most $1/(2^r - C_s)^2 \leq 1/(2^r - q)^2$, the number of unqueried valid encodings is at least

$$(2^r - C_s) \cdot P_{\text{enc}} - 2 \cdot \sqrt{(2^r - C_s) \ln(2^r - C_s)} \geq (2^r - q) \cdot P_{\text{enc}} - 2 \cdot \sqrt{r 2^r} \geq P_{\text{enc}} \cdot 2^{r-1}$$

where the last inequality holds provided $q < 2^{r-2}$ and $P_{\text{enc}} \geq 8\sqrt{r/2^r}$.

We now condition on this event. The challenger samples R uniformly without replacement until finding a valid encoding. The probability that the chosen R corresponds to one of the adversary's $E_{i,e}^+$ prior valid encoding queries is:

$$\frac{C_s^+}{C_s^+ + \#(\text{unqueried valid encodings})} \leq \frac{C_s^+}{C_s^+ + P_{\text{enc}} \cdot 2^{r-1}} \leq \frac{C_s^+}{P_{\text{enc}} \cdot 2^{r-1}}$$

Now condition on the event that the chosen R was not previously queried. In this case, the encoding is uniform among all $P_{\text{enc}} \cdot w^v$ valid encodings. The probability that it matches one

of the adversary's previous valid encoding queries at this epoch is at most $\mathbb{E}[C_s^+]/(P_{\text{enc}} \cdot w^v)$ (TODO justify).

Conclusion: Assuming $s + 1 \in \text{ord}$:

$$\Pr[\mathcal{E}_{\text{enc}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] \leq \frac{1}{(2^r - q)^2} + C_s^+ \cdot \underbrace{\left(\frac{1}{P_{\text{enc}} \cdot 2^{r-1}} + \frac{1}{P_{\text{enc}} \cdot w^v} \right)}_{\alpha}$$

6.3 Final Bound

Fix $\text{ord} \in \binom{[q+LN]}{q}$.

Let's denote $\text{ord}^- := \{s \in [0, q + LN - 1] \mid s + 1 \in \text{ord}\}$ and $\overline{\text{ord}^-} := [0, q + LN - 1] \setminus \text{ord}^-$.

$$\begin{aligned} & \sum_{s=0}^{q+LN-1} \Pr[\mathcal{E}_{\text{enc}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] = \\ & \sum_{s \in \text{ord}^-} \Pr[\mathcal{E}_{\text{enc}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] + \sum_{s \in \overline{\text{ord}^-}} \Pr[\mathcal{E}_{\text{enc}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] \\ & \leq \frac{q}{w^v} + \frac{LN}{(2^r - q)^2} + \alpha \cdot \sum_{s \in \overline{\text{ord}^-}} \mathbb{E}[C_s^+] \end{aligned}$$

Let C^+ denote the number of valid encodings found by the adversary using *fresh* (4.6) random oracle queries. We have: $\sum_{s \in \overline{\text{ord}^-}} C_s^+ \leq C^+$. Given that each random oracle query can contribute at most 1 to C^+ , with an independent success probability of P_{enc} , thus C^+ is dominated by Binomial(q, P_{enc}), so $\mathbb{E}[C^+] \leq q \cdot P_{\text{enc}}$.

We thus have: $\sum_{s \in \overline{\text{ord}^-}} \mathbb{E}[C_s^+] \leq \mathbb{E}[C^+] \leq q \cdot P_{\text{enc}}$.

Using 6.4, and the fact that $\sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\mathcal{E}_{\text{ord}}] \leq 1$ (due to 5.4) we thus have:

$$\Pr[\mathcal{E}_{\text{enc}}^{\text{found}}] \leq \frac{q}{w^v} + \frac{LN}{(2^r - q)^2} + qP_{\text{enc}} \cdot \alpha$$

We conclude:

Theorem 6.5 ($\mathcal{E}_{\text{enc}}$ Bound). *Under the constraints $q \leq 2^{r-2}$ and $P_{\text{enc}} \geq 8\sqrt{r/2^r}$:*

$$\begin{aligned} \Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{enc}}] & \leq \Pr[\mathcal{E}_{\text{enc}}^{\text{found}}] + \Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{enc}}^{\text{guessed}}] \\ & \leq \frac{q}{w^v} + \frac{LN}{(2^r - q)^2} + qP_{\text{enc}} \cdot \left[\frac{1}{P_{\text{enc}} \cdot 2^{r-1}} + \frac{1}{P_{\text{enc}} \cdot w^v} \right] + \frac{1}{w^v} \\ & \leq \frac{2q+1}{w^v} + \frac{LN}{4^{r-1}} + \frac{q}{2^{r-1}} \end{aligned}$$

7 Event $\mathcal{E}_{\text{wots}}$: WOTS Forgery

Definition 7.1. For $s \in [0, q + LN]$, we denote by $\mathcal{E}_{\text{wots}}^{\text{foundA}}(s)$, $\mathcal{E}_{\text{wots}}^{\text{foundB}}(s)$ and $\mathcal{E}_{\text{wots}}^{\text{foundC}}(s)$ the following events:

Assuming $\mathcal{E}_2$, there exists in $\mathcal{Q}(s)$ a random oracle query of the form $\text{query} = (\text{PP} \parallel \text{D}_{\text{CHAIN}} \parallel e \parallel j^* \parallel k^* \parallel x^*)$ such that PP is the public parameter of signer i , and if we denote by $(c_{i,e,j,k})_{(j,k) \in [0,v) \times [0,w)}$ his WOTS chain values at epoch e (as computed by WOTS.KeyGen in Algorithm 2) either:

- $\mathcal{Q}(s)$ contains the signature request to signer i for epoch e , and if we denote by $(x_0, \dots, x_{v-1})$ the corresponding encoding, we have $H(\text{query}) = c_{j,k+1}$ and either:
 - $\mathcal{E}_{\text{wots}}^{\text{foundA}}(s) : k^* < x_j$ (hidden preimage)

- $\mathcal{E}_{\text{wots}}^{\text{foundB}}(s) : k^* \geq x_j$ but $x^* \neq c_{j,k}$ (different revealed preimage)
- $\mathcal{E}_{\text{wots}}^{\text{foundC}}(s) : \mathcal{Q}(s)$ does not contain the signature request to signer i for epoch e , and $H(\text{query}) = c_{j^*,k^*+1}$ (premonitory preimage).

For simplicity, we denote $\mathcal{E}_{\text{wots}}^{\text{foundA}} = \mathcal{E}_{\text{wots}}^{\text{foundA}}(q + LN)$, $\mathcal{E}_{\text{wots}}^{\text{foundB}} = \mathcal{E}_{\text{wots}}^{\text{foundB}}(q + LN)$, and $\mathcal{E}_{\text{wots}}^{\text{foundC}} = \mathcal{E}_{\text{wots}}^{\text{foundC}}(q + LN)$.

Definition 7.2. We denote:

- For $s \in [0, q + LN]$, $\mathcal{E}_{\text{wots}}^{\text{found}}(s) = \mathcal{E}_{\text{wots}}^{\text{foundA}}(s) \cup \mathcal{E}_{\text{wots}}^{\text{foundB}}(s) \cup \mathcal{E}_{\text{wots}}^{\text{foundC}}(s)$
For simplicity, we denote $\mathcal{E}_{\text{wots}}^{\text{found}} = \mathcal{E}_{\text{wots}}^{\text{found}}(q + LN)$.
- $\mathcal{E}_{\text{wots}}^{\text{guessed}} = \mathcal{E}_{\text{wots}} \setminus \mathcal{E}_{\text{wots}}^{\text{found}}$

7.1 Uniformity of Hidden Values

Definition 7.3 (Revealed chain values). For $s \in [0, q + LN]$, $i \in [0, N]$, $e \in [0, L]$, $j \in [0, v]$, $k \in [0, w]$, we define the following random variables:

- $\text{Revealed}_{i,e,j,k}(s) = 1$ if there exists a signature request $\text{SIGrequest}(i, e, m) \in \mathcal{Q}(s)$ with encoding $(x_0, \dots, x_{v-1})$ such that $x_j \leq k$ (i.e., chain value $c_{i,e,j,k}$ is computable from the WOTS signature via `chain` as in Theorem 3.4); otherwise $\text{Revealed}_{i,e,j,k}(s) = 0$.
- $X_{i,e,j,k}(s) = \{x \in \{0, 1\}^n \mid \text{ROquery}(\text{PP}_i \parallel \text{D}_{\text{CHAIN}} \parallel e \parallel j \parallel k \parallel x) \in \mathcal{Q}(s)\}$
- $Y_{i,e,j,k}(s) = \{y \in \{0, 1\}^n \mid \exists x, \underbrace{\text{ROquery}(\text{PP}_i \parallel \text{D}_{\text{CHAIN}} \parallel e \parallel j \parallel k - 1 \parallel x)}_{\alpha} \in \mathcal{Q}(s) \text{ with } y = H(\alpha)\}$
(empty when $k = 0$)
- $Z_{i,e,j,k}(s) = X_{i,e,j,k}(s) \cup Y_{i,e,j,k}(s)$

As previously, we denote by: $X_{i,e,j,k} = X_{i,e,j,k}(q + LN)$, $Y_{i,e,j,k} = Y_{i,e,j,k}(q + LN)$ and $Z_{i,e,j,k} = Z_{i,e,j,k}(q + LN)$.

Lemma 7.4. For $s \in [0, q + LN]$, $i \in [0, N]$, $e \in [0, L]$, $j \in [0, v]$, $k \in [0, w - 1]$:

$$(\text{Revealed}_{i,e,j,k}(s) = 1) \subset (\text{Revealed}(i, e, j, k + 1, s) = 1)$$

Proof. Obvious from the definition of `Revealed`. □

Lemma 7.5. For $s \in [0, q + LN]$, $i \in [0, N]$, $e \in [0, L]$, $j \in [0, v]$, $k \in [0, w - 1]$:

$$|Z_{i,e,j,k}(s)| \leq q$$

Proof. Each random oracle query can contribute at most 1 to $X_{i,e,j,k}(s)$ or $Y_{i,e,j,k}(s)$, but not both. □

Lemma 7.6 (Uniformity of secret chain values). Fix $s \in [0, q + LN]$. Assume $\mathcal{E}_2$ and $\neg \mathcal{E}_{\text{wots}}^{\text{found}}(s)$. For $i \in [0, N]$, $e \in [0, L]$, $j \in [0, v]$, $k \in [0, w]$, conditioned on $\neg \text{Revealed}_{i,e,j,k}(s)$, the random variable $c_{i,e,j,k}$ is uniformly distributed over $\{0, 1\}^n \setminus [Z_{i,e,j,k}(s)]$.

Proof. Assume $\mathcal{E}_2$ and $\neg \mathcal{E}_{\text{wots}}^{\text{found}}$. We proceed by induction on s .

Base case

Before any interaction, the WOTS secret key $c_{i,e,j,0}$ is sampled uniformly at random from $\{0, 1\}^n$. All subsequent chain values $((c_{i,e,j,k})_{k=1}^{w-1})$ are determined by iteratively applying a random function, and thus also are uniform. Given $Z_{i,e,j,k}(0) = \emptyset$, the claim holds.

Inductive step (random oracle query)

Assume the lemma holds up to s and assume the $(s+1)^{\text{th}}$ query is a random oracle query: $\text{ROquery}(\text{query})$.

Case 1: If query is not of the form $(\text{PP}_i \| \text{D}_{\text{CHAIN}} \| e \| j \| k \| x)$ for any valid parameters, then by domain separation, this query is independent of all chain computations, and the conditional distribution of unrevealed chain values remains unchanged.

Case 2: If $\text{query} = (\text{PP}_i \| \text{D}_{\text{CHAIN}} \| e \| j \| k \| x)$ with valid parameters (in particular $k < w-1$), consider the chain value $c_{i,e,j,k}$.

Subcase 2-1: If $\text{Revealed}_{i,e,j,k}(s) = 1$, then $\text{Revealed}(s, i, e, j, \min(k+1, w-1)) = 1$ so the modification of $X_{i,e,j,k}$ and $Y_{i,e,j,k+1}$ due to this query does not concern chain values in the lemma's reach. All remaining chain value distribution keep their distribution by domain separation.

Subcase 2-2: Otherwise $\text{Revealed}_{i,e,j,k}(s) = 0$:

- If $x = c_{i,e,j,k}$: (adversary found a secret value) This triggers $\mathcal{E}_{\text{wots}}^{\text{found}}(s+1)$, making the lemma trivially true at the next iteration (because the lemma assumes $\neg \mathcal{E}_{\text{wots}}^{\text{found}}(s)$).
- If $x \neq c_{i,e,j,k}$: By the inductive hypothesis, $c_{i,e,j,k}$ was uniform over $\{0, 1\}^n \setminus Z_{i,e,j,k}(s)$. Conditioned on $x \neq c_{i,e,j,k}$, $c_{i,e,j,k}$ remains uniform over $\{0, 1\}^n \setminus [Z_{i,e,j,k}(s) \cup \{x\}]$, and the induction relation is still satisfied after updating: $X_{i,e,j,k}(s+1) \leftarrow X_{i,e,j,k}(s) \cup \{x\}$.

Now, assuming $c_{i,e,j,k+1}$ has not been revealed (otherwise nothing to prove about it), let's denote $y = H(\text{query})$:

- If $y = c_{i,e,j,k+1}$: (adversary found a second preimage) This triggers $\mathcal{E}_{\text{wots}}^{\text{found}}(s+1)$, making the lemma trivially true at the next iteration.
- If $y \neq c_{i,e,j,k+1}$: By the inductive hypothesis, $c_{i,e,j,k+1}$ was uniform over $\{0, 1\}^n \setminus Z_{i,e,j,k+1}(s)$. Conditioned on $y \neq c_{i,e,j,k+1}$, $c_{i,e,j,k+1}$ remains uniform over $\{0, 1\}^n \setminus [Z_{i,e,j,k+1}(s) \cup \{y\}]$, and the induction relation is still satisfied after updating: $Y_{i,e,j,k+1}(s+1) \leftarrow Y_{i,e,j,k+1}(s) \cup \{y\}$.

Inductive step (signature request)

Assume the lemma holds up to s and assume the $(s+1)^{\text{th}}$ query is a signature request: $\text{SIGrequest}(i, e, m)$. Assume $\neg \mathcal{E}_{\text{wots}}^{\text{found}}(s+1)$ holds (otherwise the lemma is trivially true). Denote by $(x_0, \dots, x_{v-1})$ the corresponding encoding for $\text{SIGrequest}(i, e, m)$. It remains to be shown that for $j \in [0, v)$ and $k < x_j$, conditioning on $(c_{i,e,j,x_j} = c)$ for some $c \in \{0, 1\}^n \setminus Z_{i,e,j,x_j}(s)$ preserves the uniform distribution of $c_{i,e,j,k}$ over $S_k := \{0, 1\}^n \setminus Z_{i,e,j,k}(s) = \{0, 1\}^n \setminus Z_{i,e,j,k}(s+1)$.

Since $\neg \mathcal{E}_{\text{wots}}^{\text{foundC}}$ holds, the adversary has never queried inputs or received outputs corresponding to intermediate chain values $c_{i,e,j,k}, c_{i,e,j,k+1}, \dots, c_{i,e,j,x_j-1}$. For any $u \in S_k$, let $p(u)$ denote the probability (from the adversary's view just before his $(s+1)^{\text{th}}$ query) that a chain of $x_j - k$ random oracle evaluations starting from u terminates at the observed value $c = c_{i,e,j,x_j}$. By symmetry of a random function on unqueried inputs, $p(u) = p$ is identical for all $u \in S_k$.

Denoting $\mathcal{T} = \mathcal{E}_2 \cap \neg \mathcal{E}_{\text{wots}}^{\text{found}}(s) \cap \neg \text{Revealed}_{i,e,j,k}(s)$, using the induction hypothesis:

$$\forall u \in S_k, \Pr[c_{i,e,j,k} = u \mid \mathcal{T}] = 1/|S_k|$$

We have by Bayes' theorem:

$$\begin{aligned} & \Pr[c_{i,e,j,k} = u \mid (c_{i,e,j,x_j} = c) \cap \mathcal{T}] \\ &= \frac{\Pr[c_{i,e,j,k} = u \mid \mathcal{T}] \cdot \Pr[c_{i,e,j,x_j} = c \mid (c_{i,e,j,k} = u) \cap \mathcal{T}]}{\Pr[c_{i,e,j,x_j} = c \mid \mathcal{T}]} \\ &= \frac{(1/|S_k|) \cdot p}{\sum_{u' \in S_k} (1/|S_k|) \cdot p} = \frac{1}{|S_k|} \end{aligned}$$

Since signature requests do not modify any of the $(Z_{i,e,j,k}(s+1))_{i,e,j,k}$, the induction property is preserved. $\square$

7.2 Guessing Probability

Lemma 7.7. $\Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{wots}}^{\text{guessed}}] \leq \frac{w}{2^n - q}$.

Proof. We use the same notations as in Section 5.6.

By the definition of $\mathcal{E}_{\text{wots}}$, there exists an index $j \in \{0, \dots, v-1\}$ such that $x_j^* < x_j$ (by incomparable encodings) (*case A*) or $\text{sig}_j^* \neq \text{sig}_j$ and $x_j^* = x_j$ (same encodings, different WOTS signature) (*case B*).

- *case A:* Using 7.6, just before outputting its forgery c_{j,x_j^*} is uniform in $\{0, 1\}^n \setminus [Z_{i,e,j,x_j^*}]$ from the adversary's perspective. Hence, sig^* collides with c_{j,x_j^*} with probability at most $1/(2^n - q)$. Assuming it doesn't, in each of the subsequent $w - 1 - x_j^*$ steps, there is (at most) a $1/2^n$ probability of collision with the next chain value. Using the union bound, the total probability of success in *case A* is at most: $1/(2^n - q) + (w - 1 - x_j^*)/2^n \leq w/(2^n - q)$.
- *case B:* Similar argument, given the fact that $\text{sig}^* \neq c_{j,x_j}$, each of the $w - 1 - x_j$ subsequent steps has (at most) a $1/2^n$ probability of collision with the authentic chain value.

$\square$

7.3 Transition Probability

Definition 7.8. We denote: $\mathcal{E}_{\text{wots}}^{\text{transition}}(s) = \neg \mathcal{E}_{\text{wots}}^{\text{found}}(s) \cap \mathcal{E}_{\text{wots}}^{\text{found}}(s+1)$ for $s \in [0, q + LN]$.

Lemma 7.9. $\mathcal{E}_{\text{wots}}^{\text{found}} = \bigcup_{s=0}^{q+LN-1} \mathcal{E}_{\text{wots}}^{\text{transition}}(s)$

Proof. $\mathcal{E}_{\text{wots}}^{\text{found}}(0) = \emptyset$ and $\forall s \in [0, q + LN - 1], \mathcal{E}_{\text{wots}}^{\text{found}}(s) \subset \mathcal{E}_{\text{wots}}^{\text{found}}(s+1)$. $\square$

Lemma 7.10.

$$\Pr[\mathcal{E}_{\text{wots}}^{\text{found}}] \leq \sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\mathcal{E}_{\text{ord}}] \cdot \sum_{s=0}^{q+LN-1} \Pr[\mathcal{E}_{\text{wots}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}]$$

Proof. c.f. 6.2 $\square$

Fix $\text{ord} \in \binom{[q+LN]}{q}$, $s \in [0, q + LN - 1]$ and assume $\mathcal{E}_2 \cap \neg \mathcal{E}_{\text{wots}}^{\text{found}}(s) \cap \mathcal{E}_{\text{ord}}$ holds.

If $s+1 \in \text{ord}$ (Random Oracle Query)

For $\mathcal{E}_{\text{wots}}^{\text{found}}(s+1)$ to be triggered, the $(s+1)^{\text{th}}$ query should have the form $\text{ROquery}(\text{PP} \parallel \text{D}_{\text{CHAIN}} \parallel e \parallel j \parallel k \parallel x)$.

Case A: $\forall m \in \{0, 1\}^n, \text{SIGrequest}(i, e, m) \notin \mathcal{Q}(s)$ (the corresponding signature has not been requested yet)

Using Theorem 7.6, $c_{i,e,j,k}$ is uniform over $\{0, 1\}^n \setminus Z_{i,e,j,k}$. With probability at most $1/(2^n - |Z_{i,e,j,k}|) \leq 1/(2^n - q)$, the query hits the secret chain value: $x = c_{i,e,j,k}$ (triggering $\mathcal{E}_{\text{wots}}^{\text{foundC}}(s+1)$).

Otherwise, $x \neq c_{i,e,j,k}$, and the output y is uniform over $\{0, 1\}^n$. The probability that $y = c_{i,e,j,k+1}$ (triggering $\mathcal{E}_{\text{wots}}^{\text{foundC}}(s+1)$) is $1/2^n$.

Overall, $\mathcal{E}_{\text{wots}}^{\text{found}}(s+1)$ is triggered with probability:

$$p_A \leq \frac{1}{2^n - q} + \frac{1}{2^n} \leq \frac{2}{2^n - q}$$

Case B: Otherwise, the signature has previously been requested: $\exists m \in \{0, 1\}^n, \text{SIGrequest}(i, e, m) \in \mathcal{Q}(s)$, with encoding $(x_0, \dots, x_{v-1})$.

Sub-case B1: $k < x_j$ (The chain value $c_{i,e,j,k}$ is unrevealed)

Using Theorem 7.6, with probability $1/(2^n - |Z_{i,e,j,k}|) \leq 1/(2^n - q)$, we have $x = c_{i,e,j,k}$, and the hash output y equals $c_{i,e,j,k+1}$, triggering $\mathcal{E}_{\text{wots}}^{\text{foundA}}(s+1)$.

Otherwise, $x \neq c_{i,e,j,k}$, and the output y is uniform over $\{0, 1\}^n$. The probability that $y = c_{i,e,j,k+1}$ (triggering $\mathcal{E}_{\text{wots}}^{\text{foundA}}(s+1)$) is $1/2^n$.

Overall, the probability of triggering $\mathcal{E}_{\text{wots}}^{\text{found}}(s+1)$ is:

$$p_{B1} \leq \frac{1}{2^n - q} + \frac{1}{2^n} \leq \frac{2}{2^n - q}$$

Sub-case B2: $k \geq x_j$ (The chain value $c_{i,e,j,k}$ has been revealed)

If $x = c_{i,e,j,k}$, then $\mathcal{E}_{\text{wots}}^{\text{foundB}}(s+1)$ is not triggered by this query. If $x \neq c_{i,e,j,k}$, the output y is uniform over $\{0, 1\}^n$, and the probability that $(y = c_{i,e,j,k+1})$ (triggering $\mathcal{E}_{\text{wots}}^{\text{foundB}}(s+1)$) is $1/2^n$.

Overall, the probability of triggering $\mathcal{E}_{\text{wots}}^{\text{found}}(s+1)$ is:

$$p_{B2} \leq \frac{1}{2^n}$$

Conclusion: Since A, B1 and B2 sub cases are disjoint, assuming $s+1 \in \text{ord}$, $\Pr[\mathcal{E}_{\text{wots}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] \leq 2/(2^n - q)$.

If $s+1 \notin \text{ord}$ (Signature Request)

Let (PP, e) respectively denote the public parameter and epoch corresponding to the $(s+1)^{\text{th}}$ query (which is a signature request).

Since $\neg \mathcal{E}_{\text{wots}}^{\text{foundC}}(s)$, no preimage of any of the $c_{i,e,j,k}$ has previously been found by the adversary, making it impossible to trigger $\mathcal{E}_{\text{wots}}^{\text{foundA}}(s+1)$ nor $\mathcal{E}_{\text{wots}}^{\text{foundB}}(s+1)$. Finally, If by contradiction we had $\mathcal{E}_{\text{wots}}^{\text{foundC}}(s+1)$, then (using the notation from 7.1) we could reuse the same c_{j^*, k^*} to show that $\mathcal{E}_{\text{wots}}^{\text{foundC}}(s)$ held, contradicting our assumption.

Conclusion: Assuming $s+1 \notin \text{ord}$, $\Pr[\mathcal{E}_{\text{wots}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] = 0$.

7.4 Final Bound

Fix $\text{ord} \in \binom{[q+LN]}{q}$.

$$\sum_{s=0}^{q+LN-1} \Pr[\mathcal{E}_{\text{wots}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] \leq \frac{2q}{2^n - q}$$

Using Theorem 7.10, and the fact that $\sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\mathcal{E}_{\text{ord}}] \leq 1$ (due to Theorem 5.4) we thus have:

$$\Pr[\mathcal{E}_{\text{wots}}^{\text{found}}] \leq \frac{2q}{2^n - q}$$

We conclude:

Theorem 7.11 ($\mathcal{E}_{\text{wots}}$ Bound).

$$\Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{wots}}] \leq \Pr[\mathcal{E}_{\text{wots}}^{\text{found}}] + \Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{wots}}^{\text{guessed}}] \leq \frac{2q}{2^n - q} + \frac{w}{2^n - q} = \frac{2q + w}{2^n - q}$$

8 Event $\mathcal{E}_{\text{leaf}}$: Merkle Leaf Collision

Definition 8.1. For $s \in [0, q + LN]$, we denote by $\mathcal{E}_{\text{leaf}}^{\text{found}}(s)$ the following event:

$\mathcal{E}_2$ holds and there exists in $\mathcal{Q}(s)$ a random oracle query of the form $\text{ROquery}(\text{PP} \parallel \text{D}_{\text{LEAF}} \parallel e \parallel \text{pk}^*)$ such that PP is the public parameter of signer i , and if we denote by $\text{pk} = (\text{pk}_0, \dots, \text{pk}_{v-1})$ the authentic WOTS public key of signer i at epoch e , we have $\text{pk}^* \neq \text{pk}$ but:

$$\text{ComputeLeaf}(\text{PP}, e, \text{pk}^*) = \text{ComputeLeaf}(\text{PP}, e, \text{pk})$$

(see Algorithm 6). For simplicity, we denote $\mathcal{E}_{\text{leaf}}^{\text{found}} = \mathcal{E}_{\text{leaf}}^{\text{found}}(q + LN)$.

Definition 8.2. We denote:

$$\bullet \mathcal{E}_{\text{leaf}}^{\text{guessed}} = \mathcal{E}_{\text{leaf}} \setminus \mathcal{E}_{\text{leaf}}^{\text{found}}$$

8.1 Guessing Probability

Lemma 8.3. $\Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{leaf}}^{\text{guessed}}] \leq 1/2^n$.

Proof. Assuming $\mathcal{E}_{\text{leaf}}^{\text{guessed}}$, the adversary outputs a forgery with a WOTS public key $\text{pk}^* \neq (\text{pk}_0 \parallel \dots \parallel \text{pk}_{v-1})$ such that the leaf hash $H(\text{PP} \parallel \text{D}_{\text{LEAF}} \parallel e \parallel \text{pk}^*)$ was never queried to the random oracle. Since the hash function is modeled as a random oracle, this unqueried value is uniform in $\{0, 1\}^n$. The probability that it equals the authentic leaf hash is exactly $1/2^n$. $\square$

8.2 Transition Probability

Definition 8.4. We denote: $\mathcal{E}_{\text{leaf}}^{\text{transition}}(s) = \neg \mathcal{E}_{\text{leaf}}^{\text{found}}(s) \cap \mathcal{E}_{\text{leaf}}^{\text{found}}(s + 1)$ for $s \in [0, q + LN - 1]$.

Lemma 8.5. $\mathcal{E}_{\text{leaf}}^{\text{found}} = \bigcup_{s=0}^{q+LN-1} \mathcal{E}_{\text{leaf}}^{\text{transition}}(s)$

Proof. $\mathcal{E}_{\text{leaf}}^{\text{found}}(0) = \emptyset$ and $\forall s \in [0, q + LN - 1], \mathcal{E}_{\text{leaf}}^{\text{found}}(s) \subset \mathcal{E}_{\text{leaf}}^{\text{found}}(s + 1)$. $\square$

Lemma 8.6.

$$\Pr[\mathcal{E}_{\text{leaf}}^{\text{found}}] \leq \sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\mathcal{E}_{\text{ord}}] \cdot \sum_{s=0}^{q+LN-1} \Pr[\mathcal{E}_{\text{leaf}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}]$$

Proof. c.f. 6.2 $\square$

Fix $\text{ord} \in \binom{[q+LN]}{q}$, $s \in [0, q + LN - 1]$ and assume $\mathcal{E}_2 \cap \neg \mathcal{E}_{\text{leaf}}^{\text{found}}(s) \cap \mathcal{E}_{\text{ord}}$ holds.

If $s + 1 \in \text{ord}$ (Random Oracle Query)

For $\mathcal{E}_{\text{leaf}}^{\text{found}}(s+1)$ to be triggered, the $(s+1)^{\text{th}}$ query must have the form $\text{ROquery}(\text{PP}_i \parallel \text{D}_{\text{LEAF}} \parallel e \parallel \text{pk}^*)$ for some signer i and epoch e , with $\text{pk}^* \neq (\text{pk}_0 \parallel \dots \parallel \text{pk}_{v-1})$ where $(\text{pk}_0, \dots, \text{pk}_{v-1})$ is the authentic WOTS public key.

Assuming $\mathcal{E}_2$, this corresponds to a *fresh* random oracle query (4.6), and thus the output $H(\text{PP}_i \parallel \text{D}_{\text{LEAF}} \parallel e \parallel \text{pk}^*)$ is uniform in $\{0, 1\}^n$. The probability that it equals the authentic leaf hash $H(\text{PP}_i \parallel \text{D}_{\text{LEAF}} \parallel e \parallel \text{pk}_0 \parallel \dots \parallel \text{pk}_{v-1})$ is $1/2^n$.

Conclusion: Assuming $s + 1 \in \text{ord}$, $\Pr[\mathcal{E}_{\text{leaf}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] \leq 1/2^n$.

If $s + 1 \notin \text{ord}$ (Signature Request)

The event $\mathcal{E}_{\text{leaf}}^{\text{found}}$ is triggered only by random oracle queries. A signature request does not add such queries to $\mathcal{Q}(s + 1)$.

Conclusion: Assuming $s + 1 \notin \text{ord}$, $\Pr[\mathcal{E}_{\text{leaf}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] = 0$.

8.3 Final Bound

Fix $\text{ord} \in \binom{[q+LN]}{q}$.

$$\sum_{s=0}^{q+LN-1} \Pr[\mathcal{E}_{\text{leaf}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] \leq \frac{q}{2^n}$$

Using Theorem 8.6, and the fact that $\sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\mathcal{E}_{\text{ord}}] \leq 1$ (due to 5.4), we have:

$$\Pr[\mathcal{E}_{\text{leaf}}^{\text{found}}] \leq \frac{q}{2^n}$$

We conclude:

Theorem 8.7 ($\mathcal{E}_{\text{leaf}}$ Bound).

$$\Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{leaf}}] \leq \Pr[\mathcal{E}_{\text{leaf}}^{\text{found}}] + \Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{leaf}}^{\text{guessed}}] \leq \frac{q}{2^n} + \frac{1}{2^n} = \frac{q+1}{2^n}$$

9 Event $\mathcal{E}_{\text{merkle}}$: Merkle Internal Node Collision

Definition 9.1. For $s \in [0, q + LN]$, we denote by $\mathcal{E}_{\text{merkle}}^{\text{found}}(s)$ the following event:

$\mathcal{E}_2$ holds and there exists in $\mathcal{Q}(s)$ a random oracle query of the form $\text{ROQuery}(\text{PP}_i \parallel \text{D}_{\text{TREE}} \parallel d \parallel j \parallel A^* \parallel B^*)$ such that PP_i is the public parameter of signer i , and if we denote by (A, B) the authentic sibling pair at depth d and index j in the Merkle tree of signer i , we have $(A^*, B^*) \neq (A, B)$ but:

$$\text{ComputeNode}(\text{PP}_i, d, j, A^*, B^*) = \text{ComputeNode}(\text{PP}_i, d, j, A, B)$$

(see Algorithm 7). For simplicity, we denote $\mathcal{E}_{\text{merkle}}^{\text{found}} = \mathcal{E}_{\text{merkle}}^{\text{found}}(q + LN)$.

Definition 9.2. We denote:

$$\bullet \mathcal{E}_{\text{merkle}}^{\text{guessed}} = \mathcal{E}_{\text{merkle}} \setminus \mathcal{E}_{\text{merkle}}^{\text{found}}$$

9.1 Guessing Probability

Lemma 9.3. $\Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{merkle}}^{\text{guessed}}] \leq h/2^n$.

Proof. Assuming $\mathcal{E}_{\text{merkle}}^{\text{guessed}}$, the adversary outputs a forgery with an authentication path $(\text{auth}_0^*, \dots, \text{auth}_{h-1}^*)$ such that at some depth $d \in [0, h-1]$, the sibling pair $(\text{left}_d^*, \text{right}_d^*)$ differs from the authentic pair $(\text{left}_d, \text{right}_d)$, yet the corresponding internal node hash $H(\text{PP} \parallel \text{D}_{\text{TREE}} \parallel d \parallel j \parallel \text{left}_d^* \parallel \text{right}_d^*)$ was never queried to the random oracle. For such d , there is a probability of $1/2^n$ of collision with the authentic internal node hash.

By a union bound over all d possible depths where collision could occur, we obtain $\Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{merkle}}^{\text{guessed}}] \leq h/2^n$. $\square$

9.2 Transition Probability

Definition 9.4. We denote: $\mathcal{E}_{\text{merkle}}^{\text{transition}}(s) = \neg \mathcal{E}_{\text{merkle}}^{\text{found}}(s) \cap \mathcal{E}_{\text{merkle}}^{\text{found}}(s+1)$ for $s \in [0, q + LN - 1]$.

Lemma 9.5. $\mathcal{E}_{\text{merkle}}^{\text{found}} = \bigcup_{s=0}^{q+LN-1} \mathcal{E}_{\text{merkle}}^{\text{transition}}(s)$

Proof. $\mathcal{E}_{\text{merkle}}^{\text{found}}(0) = \emptyset$ and $\forall s \in [0, q + LN - 1], \mathcal{E}_{\text{merkle}}^{\text{found}}(s) \subset \mathcal{E}_{\text{merkle}}^{\text{found}}(s+1)$. $\square$

Lemma 9.6.

$$\Pr[\mathcal{E}_{\text{merkle}}^{\text{found}}] \leq \sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\mathcal{E}_{\text{ord}}] \cdot \sum_{s=0}^{q+LN-1} \Pr[\mathcal{E}_{\text{merkle}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}]$$

Proof. c.f. 6.2 □

Fix $\text{ord} \in \binom{[q+LN]}{q}$, $s \in [0, q + LN - 1]$ and assume $\mathcal{E}_2 \cap \neg \mathcal{E}_{\text{merkle}}^{\text{found}}(s) \cap \mathcal{E}_{\text{ord}}$ holds.

If $s + 1 \in \text{ord}$ (Random Oracle Query)

For $\mathcal{E}_{\text{merkle}}^{\text{found}}(s+1)$ to be triggered, the $(s+1)^{\text{th}}$ query must have the form $\text{ROquery}(\text{PP}_i \parallel \text{D}_{\text{TREE}} \parallel d \parallel j \parallel A^* \parallel B^*)$ for some signer i , depth $d \in [0, h - 1]$, and index $j \in [0, 2^{h-1-d})$, with $(A^*, B^*) \neq (A, B)$ where (A, B) is the authentic sibling pair.

Assuming $\mathcal{E}_2$, this corresponds to a *fresh* random oracle query (4.6), and thus the output $H(\text{PP}_i \parallel \text{D}_{\text{TREE}} \parallel d \parallel j \parallel A^* \parallel B^*)$ is uniform in $\{0, 1\}^n$. The probability that it equals the authentic internal node hash $H(\text{PP}_i \parallel \text{D}_{\text{TREE}} \parallel d \parallel j \parallel A \parallel B)$ is $1/2^n$.

Conclusion: Assuming $s + 1 \in \text{ord}$, $\Pr[\mathcal{E}_{\text{merkle}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] \leq 1/2^n$.

If $s + 1 \notin \text{ord}$ (Signature Request)

The event $\mathcal{E}_{\text{merkle}}^{\text{found}}$ is triggered only by random oracle queries. A signature request does not add such queries to $\mathcal{Q}(s + 1)$.

Conclusion: Assuming $s + 1 \notin \text{ord}$, $\Pr[\mathcal{E}_{\text{merkle}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] = 0$.

9.3 Final Bound

Fix $\text{ord} \in \binom{[q+LN]}{q}$.

$$\sum_{s=0}^{q+LN-1} \Pr[\mathcal{E}_{\text{merkle}}^{\text{transition}}(s) \mid \mathcal{E}_{\text{ord}}] \leq \frac{q}{2^n}$$

Using Theorem 9.6, and the fact that $\sum_{\text{ord} \in \binom{[q+LN]}{q}} \Pr[\mathcal{E}_{\text{ord}}] \leq 1$ (due to 5.4), we have:

$$\Pr[\mathcal{E}_{\text{merkle}}^{\text{found}}] \leq \frac{q}{2^n}$$

We conclude:

Theorem 9.7 ($\mathcal{E}_{\text{merkle}}$ Bound).

$$\Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{merkle}}] \leq \Pr[\mathcal{E}_{\text{merkle}}^{\text{found}}] + \Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_{\text{merkle}}^{\text{guessed}}] \leq \frac{q}{2^n} + \frac{h}{2^n} = \frac{q+h}{2^n}$$

10 Main Result

Theorem 10.1 (XMSS Multi-User SUF-CMA Security). *Let $\mathcal{A}$ be a (classical) adversary against XMSS in the multi-user SUF-CMA game, making q random oracle queries. Conditioned on the event in which all signer's public parameter are distinct (which happens with probability $1 - \Pr[\text{PP collision}] \geq 1 - N^2/2^{\text{pp}+1}$, independently from the adversary), the probability that the adversary wins the SUF-CMA game satisfies:*

$$\begin{aligned} \Pr[\mathcal{A} \text{ wins}] &\leq \Pr[R \text{ not found}] + \Pr[\mathcal{A} \text{ wins} \cap \mathcal{E}_2] \\ &\leq \Pr[R \text{ not found}] + \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_{\text{enc}}] + \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_{\text{wots}}] \\ &\quad + \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_{\text{leaf}}] + \Pr[\mathcal{A} \text{ wins} \mid \mathcal{E}_{\text{merkle}}] \end{aligned}$$

Under the constraints $q \leq 2^{r-2}$, $q \leq 2^{n-1}$ and $P_{\text{enc}} \geq 8\sqrt{r/2^r}$:

$$\begin{aligned} \Pr[\mathcal{A} \text{ wins}] &\leq LN(1 - P_{\text{enc}})^{2^r} + \left(\frac{2q+1}{w^v} + \frac{LN}{4^{r-1}} + \frac{q}{2^{r-1}} \right) + \left(\frac{2q+w}{2^n - q} \right) + \left(\frac{q+1}{2^n} \right) + \left(\frac{q+h}{2^n} \right) \\ &\leq LN(1 - P_{\text{enc}})^{2^r} + \frac{2q+1}{w^v} + \frac{LN}{4^{r-1}} + \frac{q}{2^{r-1}} + \frac{6q+2w+h+1}{2^n} \end{aligned}$$

Assuming $n = r$, we obtain:

$$\Pr[\mathcal{A} \text{ wins}] \leq LN(1 - P_{\text{enc}})^{2^r} + \frac{2q+1}{w^v} + \frac{LN}{4^{r-1}} + \frac{8q+2w+h+1}{2^n}$$

Proof. Direct application of Theorems 5.1, 6.5, 7.11, 8.7 and 9.7 and sections 5.3 to 5.5, using Theorem 5.2 to ensure exhaustiveness. $\square$

11 Concrete Parameter Selection

We use the lifetime: $L = 2^{32}$.

We use $r = n$, i.e. as many bits of randomness (for encoding) as hash output size.

We always chose w and v such that $P_{\text{enc}} \approx 2^{-9}$, i.e. 9 bits of grinding at signing to find a valid randomness.

We recommend to choose public parameters of size $\text{pp} = 123$ bits. For instance, the probability of collision of PP among $N = 2^{30}$ signers is $\leq 2^{-62}$. Even in case of collision, we adapt the previous section to show that number of lost security bits is small (think less than 4 bits), except, again, with very low probability.

We present concrete bounds when the hash function is derived from Poseidon2 [4] over 16 field elements in $\mathbb{F}_p$, with $p = 2^{31} - 2^{24} + 1$ (koala-bear prime). In this setting, we conservatively assume that hashing the WOTS public key requires v hashes (even though we can do better, but this would require further explanation (TODO)). Finally, we assume the encoding can be computed in 3 hashes.

11.1 150-bit security

- Hash output: $n \geq 154.5$ bits $\rightarrow$ 5 field elements
- WOTS parameters: $v \cdot \log_2 w \geq 152$ bits
- **Lightweight**
 - $w = 16$, $v = 38$
 - target sum $t = 341$
 - 302 hashes per XMSS verification
 - Signature size: 1376 bytes
- **Fewer hashes**
 - $w = 8$, $v = 51$
 - target sum $t = 215$
 - 228 hashes per XMSS verification
 - Signature size: 1628 bytes

11.2 128-bit security

- Hash output: $n \geq 132$ bits $\rightarrow$ 5 field elements
- WOTS parameters: $v \cdot \log_2 w \geq 130$ bits
- **Lightweight**
 - $w = 16, v = 33$
 - target sum $t = 301$
 - 262 hashes per XMSS verification
 - Signature size: 1279 bytes
- **Fewer hashes**
 - $w = 8, v = 44$
 - target sum $t = 188$
 - 199 hashes per XMSS verification
 - Signature size: 1492 bytes

12 TODO

- allow some adversary queries before keygen
- analysis for colliding public parameters?
- quantum adversary
- explicit construction of the encoding (currently treated as a black box). Proof of security even in case of non-uniform encoding (key property: probability of the most likely encoding $\leq 1/2^{\text{security level}}$)
- explicit construction of the compression of the WOTS public key (currently treated as a black box).
- Even tighter bounds? We can likely gain 2 or 3 bits of security by refining some of the union bounds, as well as increasing the number of signers.

A Annex: Technical Results

A.1 Birthday Bound

Lemma A.1 (Birthday Bound). *If k values are sampled independently and uniformly at random from a set of size n , the probability that at least two values are equal is at most $k^2/(2n)$.*

Proof. Let $X_1, \dots, X_k$ be the sampled values. By the union bound over all $\binom{k}{2}$ pairs:

$$\Pr[\exists i \neq j : X_i = X_j] \leq \sum_{1 \leq i < j \leq k} \Pr[X_i = X_j] = \binom{k}{2} \cdot \frac{1}{n} = \frac{k(k-1)}{2n} \leq \frac{k^2}{2n}. \quad \square$$

A.2 Binomial Concentration

Lemma A.2 (Binomial Concentration). *Let $X \sim \text{Binomial}(n, p)$. For any $\delta > 0$:*

$$\Pr(|X - np| \geq \delta) \leq 2 \exp\left(-\frac{2\delta^2}{n}\right)$$

In particular, with probability at least $1 - 1/n^2$:

$$|X - np| \leq \sqrt{\frac{n \ln(2n^2)}{2}} \leq 2\sqrt{n \ln n}$$

Proof. Write $X = \sum_{i=1}^n X_i$ where $X_1, \dots, X_n$ are independent Bernoulli(p) random variables. Hoeffding's inequality states that for independent random variables $Y_i \in [a_i, b_i]$:

$$\Pr\left(\left|\sum_{i=1}^n Y_i - \mathbb{E}\left[\sum_{i=1}^n Y_i\right]\right| \geq t\right) \leq 2 \exp\left(-\frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2}\right)$$

For Bernoulli random variables with $a_i = 0$ and $b_i = 1$:

$$\Pr(|X - np| \geq t) \leq 2 \exp\left(-\frac{2t^2}{n}\right)$$

Setting the right-hand side equal to ε and solving:

$$2 \exp\left(-\frac{2t^2}{n}\right) = \varepsilon \implies t = \sqrt{\frac{n \ln(2/\varepsilon)}{2}}$$

The stated bounds follow by substitution. Specifically, setting $\varepsilon = 1/n^2$:

$$t = \sqrt{\frac{n \ln(2n^2)}{2}} = \sqrt{\frac{n(\ln 2 + 2 \ln n)}{2}} \leq \sqrt{\frac{n \cdot 3 \ln n}{2}} \leq 2\sqrt{n \ln n}$$

for $n \geq 3$. □

References

- [1] J. Drake, D. Khovratovich, M. Kudinov, and B. Wagner, “Hash-based multi-signatures for post-quantum ethereum,” Cryptology ePrint Archive, Paper 2025/055, 2025. [Online]. Available: <https://eprint.iacr.org/2025/055>
- [2] D. Khovratovich, M. Kudinov, and B. Wagner, “At the top of the hypercube – better size-time tradeoffs for hash-based signatures,” Cryptology ePrint Archive, Paper 2025/889, 2025. [Online]. Available: <https://eprint.iacr.org/2025/889>
- [3] J. Drake, D. Khovratovich, M. Kudinov, and B. Wagner, “Technical note: LeanSig for post-quantum ethereum,” Cryptology ePrint Archive, Paper 2025/1332, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1332>
- [4] L. Grassi, D. Khovratovich, and M. Schofnegger, “Poseidon2: A faster version of the poseidon hash function,” Cryptology ePrint Archive, Paper 2023/323, 2023. [Online]. Available: <https://eprint.iacr.org/2023/323>